



Gobierno de
México

Ciencia y Tecnología
Secretaría de Ciencia, Humanidades, Tecnología e Innovación



UNRC
Universidad Nacional Roserío Castellanos
Baja California

Licenciatura en Tecnologías de la Información y la Comunicación

Estructura de Datos Segundo Semestre

Módulo 3. Introducción a estructuras no lineales Gestión de turnos de atención mediante colas dinámicas

Docente: Lic. Cristina Martínez Ruiz

Alumna: Viviana Selene Reyes Ciénega
Matricula: 25123768-1
Grupo: 208
27 de mayo 2026



1.- Descripción del Programa

1.1 Objetivo general del programa

El objetivo principal de esta solución tecnológica es implementar un sistema automatizado y eficiente para la administración y control de turnos de atención ciudadana dentro del programa social PAD-Bienestar. El software busca erradicar el desorden y los tiempos de espera prolongados mediante un flujo secuencial automatizado, garantizando un servicio equitativo, transparente y a prueba de errores humanos.

1.2 Estructura de datos utilizada

Para resolver este problema se seleccionó una **Cola Dinámica** (conocida técnicamente en el desarrollo de software como estructura de tipo *FIFO* o *Primero en Entrar, Primero en Salir*). A diferencia de las listas fijas (arreglos estáticos) que tienen un límite predefinido de espacio, esta estructura se construye utilizando nodos independientes eslabonados mediante punteros (direcciones de memoria física). La cola está gobernada de forma estricta por dos punteros de control: el puntero frente, que vigila la salida del próximo elemento a procesar, y el puntero final, que coordina la integración de los nuevos elementos.

1.3 Relación entre la Estructura de Datos y el Problema de Gestión de Turnos:

Existe una correspondencia exacta entre la lógica de operación de las oficinas del PAD-Bienestar y el comportamiento de una cola dinámica.

En mi experiencia en la agencia de publicidad, esto equivale exactamente a la organización de la fila para el servicio de alimentación o el registro de extras en una convocatoria de actuación (*casting*). El primer actor que llega y se registra es estrictamente el primero en recibir su comida o pasar a su prueba; las personas rezagadas se forman obligatoriamente al final. No se permite que nadie se sitúe en medio de la fila de espera ni que se altere el orden de llegada. Esto garantiza un control administrativo impecable y equitativo.

Desarrollo

Enlace al repositorio GitHub <https://github.com/artemusicayvideo-gif/Estructura-de-Datos-Modulo-3>



2.- Análisis de Resultados (Lógica del Sistema)

2.1 Comportamiento en Memoria al Agregar un Turno (Asignación Dinámica):

Cuando un operador registra un nuevo turno en el sistema (Opción 1), se invoca la función `agregarTurno`. En este instante, el sistema operativo ejecuta una operación de asignación de memoria mediante el operador dinámico `new` de C++.

```
// 3. AGREGAR TURNO A LA COLA
void agregarTurno(Turno** frente, Turno** final, int numero) {
    if (!validarTurno(numero)) {
        cout << "-> ✖ Error: El numero de turno debe ser mayor que 0." << endl;
        return;
    }

    // Se pide memoria a la computadora para el nuevo turno
    Turno* nuevo = new Turno;
    nuevo->numeroTurno = numero;
    nuevo->siguiente = nullptr; // Como acaba de llegar, nadie va detras de el
```

La computadora reserva un bloque físico de memoria RAM del tamaño exacto de nuestra estructura. El programa realiza entonces una validación crucial: si la cola se encuentra vacía, este nuevo nodo se convierte de forma simultánea en el `frente` y en el `final` de la fila. Si ya existen elementos previos, el sistema redirige el enlace del antiguo último nodo hacia la dirección del nuevo turno y actualiza el indicador final.

```
// AQUI APLIQUE LA INSTRUCCIÓN CLAVE QUE LA PROFESORA DIO EN CLASE
// Si la cola está vacía, este nodo es el primero y también el último
if (*frente == nullptr) {
    *frente = nuevo;
    *final = nuevo;
} else {
    // Si ya hay gente, le decimos al que estaba al final que alguien llego detras
    (*final)->siguiente = nuevo;
    // Actualizamos nuestro apuntador para que el nuevo sea oficialmente el final
    *final = nuevo;
}
```

2.2 Comportamiento en Memoria al Atender un Turno (Liberación Dinámica):

Al seleccionar la opción de atender (Opción 2), el sistema aplica estrictamente el principio de justicia distributiva (*FIFO*). El programa no realiza búsquedas innecesarias ni le pregunta al operador a quién desea borrar; automáticamente toma al nodo apuntado por el indicador `frente`.



Para evitar la pérdida de continuidad en el sistema, se crea un puntero temporal que retiene al nodo actual mientras el puntero **frente** avanza pacientemente hacia el siguiente turno de la lista. Una vez hecho este puenteo, se ejecuta el operador **delete**. Esta instrucción destruye físicamente el nodo atendido y le devuelve ese espacio de memoria RAM a la computadora, evitando lo que en la informática se conoce como fugas de memoria o retención innecesaria de recursos.

```
// 4. ATENDER TURNO (Respeta el orden FIFO: Primero en llegar, primero en salir)
void atenderTurno(Turno** frente, Turno** final) {
    if (*frente == nullptr) {
        cout << " ⚠ No hay turnos en espera. La cola esta vacia." << endl;
        return;
    }

    // Identificamos al primero de la fila
    Turno* auxiliar = *frente;
    cout << " 🏠 Atendiendo al turno " << auxiliar->numeroTurno << "." << endl;

    // Recorremos la fila hacia adelante
    *frente = (*frente)->siguiente;

    // Si al recorrer la fila, resulta que ya no hay nadie, limpiamos el final
    if (*frente == nullptr) {
        *final = nullptr;
    }

    // Liberamos la memoria (Evita fugas de memoria)
    delete auxiliar;
}
```

2.3 Recorrido de la Cola para Visualización:

Para mostrar los turnos en espera (Opción 3), la función **mostrarCola** crea un puntero de inspección llamado **actual**, el cual se posiciona al inicio (**frente**). Mediante un ciclo condicional **while**, el puntero va "saltando" de nodo en nodo siguiendo la dirección grabada en el campo **siguiente**. El recorrido avanza de manera limpia imprimiendo los folios en pantalla hasta encontrar un valor nulo (**nullptr**), el cual indica formalmente que hemos llegado al final de la fila de espera.



```
// 5. MOSTRAR COLA DE TURNOS
void mostrarCola(Turno* frente) {
    Turno* actual = frente;

    if (actual == nullptr) {
        cout << " 🚪 No hay turnos en espera. La cola esta vacia." << endl;
        return;
    }

    cout << "\n 🚪 Cola de turnos en espera 🕒:" << endl;
    while (actual != nullptr) {
        cout << "[Turno " << actual->numeroTurno << "] -> ";
        actual = actual->siguiente;
    }
    cout << " 🚪 FIN DE LA FILA 🟡 " << endl;
}
```

2.3 Mantenimiento del Orden FIFO:

La integridad del flujo se resguarda gracias a que el código bloquea el acceso a las funciones: la inserción de nuevos turnos está restringida exclusivamente a la posición del puntero **final**, mientras que la extracción y procesamiento de bajas solo se concede a la posición del puntero **frente**. Esta restricción matemática impide cualquier alteración en el orden cronológico de atención.

3.- Evidencia de Ejecución

A continuación, se presentan las pruebas de funcionamiento del sistema PAD-Bienestar, demostrando tanto el flujo ideal de operación como la robustez del programa ante posibles errores de captura por parte de los operadores.

3.1 Menú Interactivo Principal

```
#####
SISTEMA PAD-BIENESTAR M3
#####
1. Agregar un turno
2. Atender un turno
3. Mostrar cola de turnos
4. Salir
Seleccione una opcion: █
```



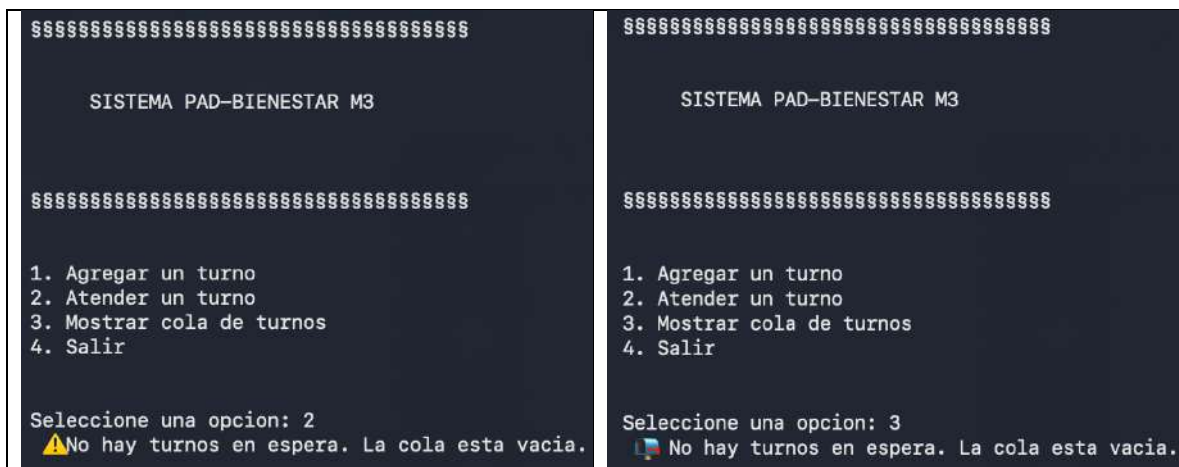

3.2 Registro de Datos (correctos): Registro de turno, atender un turno, mostrar cola de turnos y Salir

<pre>##### SISTEMA PAD-BIENESTAR M3 ##### 1. Agregar un turno 2. Atender un turno 3. Mostrar cola de turnos 4. Salir Seleccione una opcion: 1 Ingrese el numero de turno: 10 ✅ Turno 10 agregado exitosamente a la cola.</pre>	<pre>##### SISTEMA PAD-BIENESTAR M3 ##### 1. Agregar un turno 2. Atender un turno 3. Mostrar cola de turnos 4. Salir Seleccione una opcion: 1 Ingrese el numero de turno: 30 ✅ Turno 30 agregado exitosamente a la cola.</pre>
<pre>##### SISTEMA PAD-BIENESTAR M3 ##### 1. Agregar un turno 2. Atender un turno 3. Mostrar cola de turnos 4. Salir Seleccione una opcion: 1 Ingrese el numero de turno: 50 ✅ Turno 50 agregado exitosamente a la cola.</pre>	<pre>##### SISTEMA PAD-BIENESTAR M3 ##### 1. Agregar un turno 2. Atender un turno 3. Mostrar cola de turnos 4. Salir Seleccione una opcion: 3 📄 Cola de turnos en espera 🕒: [Turno 10] -> [Turno 30] -> [Turno 50] -> 🚪 FIN DE LA FILA 🟡</pre>
<pre>##### SISTEMA PAD-BIENESTAR M3 ##### 1. Agregar un turno 2. Atender un turno 3. Mostrar cola de turnos 4. Salir Seleccione una opcion: 2 👤 Atendiendo al turno 10.</pre>	<pre>##### SISTEMA PAD-BIENESTAR M3 ##### 1. Agregar un turno 2. Atender un turno 3. Mostrar cola de turnos 4. Salir Seleccione una opcion: 3 📄 Cola de turnos en espera 🕒: [Turno 30] -> [Turno 50] -> 🚪 FIN DE LA FILA 🟡</pre>



3.3 Registro de Datos (erroneos)

Comportamiento del programa ante el intento de ejecutar las opciones 'Atender turno' y 'Mostrar cola' sin haber registrado usuarios previamente



Manejo de excepciones en la captura de datos: Para garantizar la estabilidad del sistema, se integró un protocolo de validación en el menú principal. Mediante los comandos `cin.clear()` y `cin.ignore()`, el programa bloquea el ingreso de caracteres no válidos (evitando que el sistema colapse) y emite una alerta visual al operador.



```
#####

SISTEMA PAD-BIENESTAR M3

#####

1. Agregar un turno
2. Atender un turno
3. Mostrar cola de turnos
4. Salir

Seleccione una opcion: s
X Opcion invalida. Intente nuevamente con un numero del 1 al 4.
```

```
#####

SISTEMA PAD-BIENESTAR M3

#####

1. Agregar un turno
2. Atender un turno
3. Mostrar cola de turnos
4. Salir

Seleccione una opcion: X
X Opcion invalida. Intente nuevamente con un numero del 1 al 4.
```

```
#####

SISTEMA PAD-BIENESTAR M3

#####

1. Agregar un turno
2. Atender un turno
3. Mostrar cola de turnos
4. Salir

Seleccione una opcion: 100
X Opcion invalida. Intente nuevamente con un numero del 1 al 4.
```

Conclusiones

Ventajas de las Colas Dinámicas frente a Otras Estructuras:

Las colas dinámicas ofrecen una ventaja insuperable en la gestión de flujos de servicios en comparación con los arreglos tradicionales. Mientras que un arreglo estático obliga a definir un tamaño máximo desde la planeación (lo que provocaría desperdicio de memoria si van pocas personas o el rechazo de usuarios si la oficina se satura), la cola dinámica permite que el sistema "respire". El programa se expande y consume RAM únicamente cuando un ciudadano llega, y se contrae de forma inmediata cuando el ciudadano se retira.

Importancia de la Memoria Dinámica en la Solución de Problemas Reales:

En las Tecnologías de la Información y la Comunicación (TIC), la memoria dinámica representa el puente entre el software de laboratorio y las aplicaciones del mundo real. En proyectos de gran escala institucional como el PAD-Bienestar —o incluso en la gestión de presupuestos variables para producciones multimedia en agencias de publicidad—, la demanda es impredecible y oscilante. Dominar los operadores new y delete permite desarrollar sistemas estables, flexibles y altamente eficientes que cuidan los recursos de hardware de la organización.



Relación entre Estructuras de Datos y Organización Eficiente:

La arquitectura del software debe ser el espejo fiel de las reglas de operación de una institución. Elegir la estructura de datos correcta (en este caso, una cola no lineal dinámica para turnos) es la decisión de diseño más crítica en la ingeniería de software. Al alinear la lógica de la computadora con las necesidades del ciudadano, se logra una optimización real de los procesos informáticos, garantizando sistemas con un rendimiento óptimo, escalables y fáciles de mantener a largo plazo.

Referencias:

- Deitel, P. J., & Deitel, H. M. (2014). *C++ Cómo programar* (9.^a ed.). Pearson Educación.
- Joyanes Aguilar, L. (2017). *Fundamentos de programación: algoritmos, estructuras de datos y objetos* (4.a ed.). McGraw-Hill.
- Universidad Nacional Rosario Castellanos. (2026). *Módulo 3: Introducción a estructuras no lineales. Estructura de Datos*. Licenciatura en Tecnologías de la Información y la Comunicación